

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab **Theory:**

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models.
Take the words —> Give their numeric representation —> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

$TF(t) = (\text{Number of times term } t \text{ appears in a document}) / (\text{Total number of terms in the document})$

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

$IDF(t) = \log_e(\text{Total number of documents} / \text{Number of documents with term } t \text{ in it})$

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

docs = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding in python"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

bow_df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())

print("BoW Representation:\n")
print(bow_df)
```

BoW Representation:

	amazing	coding	in	is	learning	love	machine	python
0	0	0	0	0	1	1	1	0
1	1	0	0	1	1	0	1	0
2	0	1	1	0	0	1	0	1

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

2.Perform TF vectorization of the corpus

```
from sklearn.feature_extraction.text import CountVectorizer

import pandas as pd

docs = [

    "I love machine learning",

    "Machine learning is amazing",

    "I love coding in python"

]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(docs).toarray()

tf = X / X.sum(axis=1, keepdims=True)

tf_df = pd.DataFrame(tf, columns=vectorizer.get_feature_names_out())

print("TF Representation:\n")

print(tf_df)
```

TF Representation:

	amazing	coding	in	is	learning	love	machine	python
0	0.00	0.00	0.00	0.00	0.333333	0.333333	0.333333	0.00
1	0.25	0.00	0.00	0.25	0.250000	0.000000	0.250000	0.00
2	0.00	0.25	0.25	0.00	0.000000	0.250000	0.000000	0.25

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

Program (Code):

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
```

```
docs = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding in python"
]

bow = CountVectorizer()
bow_matrix = bow.fit_transform(docs)

bow_similarity = cosine_similarity(bow_matrix)

print("BoW Similarity Matrix:\n", bow_similarity)

tf = CountVectorizer()
tf_matrix = tf.fit_transform(docs).toarray()

tf_matrix = tf_matrix / tf_matrix.sum(axis=1, keepdims=True)

tf_similarity = cosine_similarity(tf_matrix)

print("\nTF Similarity Matrix:\n", tf_similarity)

tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(docs)

tfidf_similarity = cosine_similarity(tfidf_matrix)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

```
print("\nTF-IDF Similarity Matrix:\n", tfidf_similarity)
```

Results:

To be attached with

Observation:

- BoW measures similarity based on **word count only**, without considering importance of words.
- TF normalizes word frequency, giving better comparison than BoW.
- TF-IDF gives **best result** because it reduces the weight of common words and increases the importance of rare words.
- Documents 1 and 2 are more similar because both contain “**machine learning**”.
- Document 3 is less similar because it talks about **coding and python**, which are different topics.
- TF-IDF provides more meaningful similarity compared to BoW and TF.

Post Lab Exercise:

Take three documents and find similarity using

- BoW vectorization
- TF Vectorization
- TF-IDF Vectorization

Comment over the answer obtained in each of the case.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92301733023

BoW Similarity Matrix:

```
[[1.          0.57735027 0.28867513]
 [0.57735027 1.          0.          ]
 [0.28867513 0.          1.          ]]
```

TF Similarity Matrix:

```
[[1.          0.57735027 0.28867513]
 [0.57735027 1.          0.          ]
 [0.28867513 0.          1.          ]]
```

TF-IDF Similarity Matrix:

```
[[1.          0.49426499 0.23211804]
 [0.49426499 1.          0.          ]
 [0.23211804 0.          1.          ]]
```